

Group 3 - Final Backend Requirement Plan

Blood Donation & Donor Management System - Spring Boot 3.x + ReactJS + PostgreSQL

Purpose: This PDF is the working contract for backend inputs, dropdowns/comboboxes, status rules, JPA entity annotations, validation annotations, and module dependencies. It follows the lecturer specification: Spring Data JPA + Hibernate, Spring Security JWT, PostgreSQL, layered architecture, and React/Axios frontend integration.

1. Final technical decisions

Item	Decision
Backend data access	Spring Data JPA + Hibernate. Repositories extend <code>JpaRepository<Entity, Long></code> . No JDBC RowMapper plan.
Entity folder	Use entity package. Each entity class uses <code>@Entity</code> , <code>@Table</code> , <code>@Id</code> , <code>@GeneratedValue</code> , and relationship annotations.
Database	PostgreSQL database name: <code>blood_donation_db</code> . Use <code>spring.jpa.hibernate.ddl-auto=update</code> during development.
Security	Spring Security with JWT. Public endpoints: <code>/api/auth/register</code> and <code>/api/auth/login</code> . Other APIs protected.
Roles	<code>ROLE_ADMIN</code> and <code>ROLE_USER</code> . In this system, <code>ROLE_USER</code> means donor/user account.
Validation	Use Bean Validation on request DTOs: <code>@NotBlank</code> , <code>@NotNull</code> , <code>@Email</code> , <code>@Size</code> , <code>@Min</code> , <code>@Max</code> , <code>@PastOrPresent</code> , <code>@Pattern</code> .
Frontend dropdowns	Dropdown data should come from backend when it is database data. Fixed enum values can be hardcoded in frontend and validated in backend.

Do not start service logic before the entity relationships are agreed. Entities are the database contract between Isse and Yasmin.

2. Final backend package structure

This is the final structure. Use entity, not model, because the lecturer requirement explicitly asks for Entity Layer and JPA annotations.

```
src/main/java/backend
|-- auth
| |-- controller/AuthController.java
| |-- service/AuthService.java
| |-- request/RegisterRequest.java, LoginRequest.java
| `-- response/AuthResponse.java
|
|-- user
| |-- entity/User.java
| |-- repository/UserRepository.java
| |-- service/UserService.java
| `-- response/UserResponse.java
|
|-- category
| |-- controller/CategoryController.java
| |-- service/CategoryService.java
| |-- repository/CategoryRepository.java
| |-- entity/BloodCategory.java
| |-- request/CategoryRequest.java
| `-- response/CategoryResponse.java
|
|-- donor / inventory / donation / announcement
| |-- controller
| |-- service
| |-- repository
| |-- entity
| |-- request
| `-- response
|
|-- dashboard/controller, service, response
|-- report/controller, service, request, response
|-- security/JwtService, JwtAuthenticationFilter, SecurityConfig, CustomUserDetailsService
`-- exception/BadRequestException, ResourceNotFoundException, DuplicateResourceException, ErrorResponse, GlobalExceptionHandler
```

Layer rule

- Controller: receives HTTP request, uses @Valid, returns ResponseEntity.
- Service: business logic, validation, status calculation, relationship checks, @Transactional when multiple DB actions happen.
- Repository: data access only, extends JpaRepository.
- Entity: database table and relationships.
- Request: frontend input DTO. Response: clean frontend output DTO. Do not return password.

3. Global statuses and dropdown sources

These rules must be used exactly. Most project errors come from making status a manual input when it should be calculated or defaulted.

Field / Status	UI type	Allowed values	Source	Backend rule
Role	Dropdown on register/admin user form	ROLE_ADMIN, ROLE_USER	Frontend fixed list + backend enum/check	RegisterRequest validates role. Entity stores role as enum/string with @Column(nullable=false).
Blood group / category	Dropdown	A+, A-, B+, B-, AB+, AB-, O+, O-	GET /api/categories/active from blood_categories table	Frontend sends categoryId. Backend finds BloodCategory by id. Do not send random blood group.
Gender	Dropdown	MALE, FEMALE	Frontend fixed list + backend enum/check	DonorRequest validates gender. Entity stores Gender enum or string.
Inventory status	No dropdown	OUT_OF_STOCK, LOW_STOCK, AVAILABLE	Calculated by InventoryService	quantity=0 OUT_OF_STOCK; 1-3 LOW_STOCK; 4+ AVAILABLE. User never chooses it.
Donation status	No dropdown	RECORDED only	DonationService default	Every saved donation is RECORDED. No pending/approved/rejected.
Urgency	Dropdown	NORMAL, URGENT, EMERGENCY	Frontend fixed list + backend enum/check	AnnouncementRequest validates urgency.
Announcement status	Dropdown only on status update	ACTIVE, FULFILLED, CANCELLED	Frontend fixed list + backend enum/check	Create default ACTIVE. Update ACTIVE to FULFILLED or CANCELLED.
Report type	Dropdown	DONOR, INVENTORY, DONATION, ANNOUNCEMENT	Frontend fixed list + backend enum/check	ReportFilterRequest validates reportType.
Report status filter	Dropdown optional	Depends on reportType	Frontend changes options by reportType	INVENTORY uses inventory statuses; ANNOUNCEMENT uses announcement statuses; DONATION uses RECORDED; DONOR normally no status.

Best practice: for blood category dropdowns, show the user bloodGroup text but submit categoryId to backend. This avoids spelling mismatch and preserves foreign keys.

4. Auth + Security module

Auth has no repository. It uses UserRepository. Security uses JWT. Auth endpoints are public; all main module endpoints are protected.

Class	Field / method	UI / input type	Validation annotations	Rule / output
RegisterRequest	fullName	Text input	@NotBlank, @Size(min=2,max=80)	Required. Stored in User.fullName.
RegisterRequest	email	Email input	@NotBlank, @Email, @Size(max=120)	Must be unique. Duplicate -> DuplicateResourceException 409.
RegisterRequest	password	Password input	@NotBlank, @Size(min=6,max=100)	Encode before saving with PasswordEncoder.
RegisterRequest	role	Dropdown	@NotNull	Allowed: ROLE_ADMIN, ROLE_USER. Usually public register can restrict to ROLE_USER.
LoginRequest	email	Email input	@NotBlank, @Email	Find user by email.
LoginRequest	password	Password input	@NotBlank	Compare using PasswordEncoder.matches.
AuthResponse	token	Response only	N/A	JWT returned after login/register success.
AuthResponse	userId, fullName, email, role	Response only	N/A	Never return password.

Security files and jobs

File	Responsibility
SecurityConfig.java	Defines public endpoints, protected endpoints, stateless session, password encoder, JWT filter, role restrictions.
JwtService.java	Creates token, extracts username/email, validates token expiration and signature.
JwtAuthenticationFilter.java	Reads Authorization: Bearer token, validates JWT, sets authentication in SecurityContext.
CustomUserDetailsService.java	Loads user by email from UserRepository for Spring Security.

Controller annotations: @RestController, @RequestMapping("/api/auth"), @PostMapping, @RequestBody, @Valid, ResponseEntity.

5. User and Category modules

User is the account table used by authentication. Category is the source of blood group dropdowns across donor, inventory, donation, announcement, and report.

Module/class	Field	Input/UI type	Annotations	Rule
User entity	id	Generated	@Id, @GeneratedValue(strategy=IDENTITY)	Primary key users.id.
User entity	fullName	Text	@Column(nullable=false,length=80)	Comes from RegisterRequest.
User entity	email	Email	@Column(nullable=false,unique=true,length=120)	Login username. Must be unique.
User entity	password	Password	@Column(nullable=false)	Store encoded password, not plain text.
User entity	role	Dropdown	@Enumerated(EnumType.STRING) or @Column(nullable=false,length=20)	ROLE_ADMIN or ROLE_USER.
UserRepository	findByEmail	N/A	JpaRepository method	Used by login and security.
UserRepository	existsByEmail	N/A	JpaRepository method	Used by register validation.
CategoryRequest	bloodGroup	Dropdown or admin input	@NotBlank, @Pattern for valid blood group	Allowed 8 groups only.
CategoryRequest	description	Text area	@Size(max=150)	Optional.
CategoryRequest	active	Checkbox/switch	Optional Boolean	Default true.
BloodCategory entity	id	Generated	@Id, @GeneratedValue(strategy=IDENTITY)	Primary key blood_categories.id.
BloodCategory entity	bloodGroup	Text/dropdown	@Column(nullable=false,unique=true,length=3)	A+, A-, B+, B-, AB+, AB-, O+, O-.
BloodCategory entity	description, active	Text / checkbox	@Column(length=150), @Column(nullable=false)	Active categories appear in dropdowns.

Category endpoints

- GET /api/categories - admin list all.
- GET /api/categories/active - frontend combobox source for blood groups.
- POST /api/categories - admin creates category if not pre-seeded.
- PUT /api/categories/{id} - admin updates description/active.
- DELETE /api/categories/{id} - optional; prefer deactivate if related records exist.

6. Donor module

Donor profile is connected to one existing user account and one blood category. Donor package must not contain email/password fields; those belong to auth/user.

Class	Field	UI / input type	Source / options	Validation / annotations
CreateDonorRequest	userId	Dropdown/search user	Users with ROLE_USER and no donor profile	@NotNull. Service: user must exist and role must be ROLE_USER.
CreateDonorRequest	categoryId	Blood group dropdown	GET /api/categories/active	@NotNull. Service: category must exist and active=true.
CreateDonorRequest	phone	Text input	User typed	@NotBlank, @Size(max=20). Optional @Pattern for phone.
CreateDonorRequest	address	Text input	User typed	@NotBlank, @Size(min=2,max=120).
CreateDonorRequest	age	Number input	User typed	@NotNull, @Min(18), @Max(65).
CreateDonorRequest	gender	Dropdown	MALE, FEMALE	@NotNull. Store as Gender enum or string.
CreateDonorRequest	weight	Number input	User typed	@Positive if provided. Optional.
Donor entity	id	Generated	N/A	@Id, @GeneratedValue(strategy=ID ENTITY).
Donor entity	user	Relationship	users.id	@OneToOne, @JoinColumn(name="user_id", nullable=false, unique=true).
Donor entity	category	Relationship	blood_categories.id	@ManyToOne, @JoinColumn(name="category_id", nullable=false).
Donor entity	phone/address/age/gender/weight	Columns	N/A	@Column constraints; @Enumerated(EnumType.STRING) for gender if enum used.

Donor endpoints

- POST /api/donors - create donor profile.
- GET /api/donors - list/search donors.
- GET /api/donors/{id} - donor details.
- PUT /api/donors/{id} - update donor profile.
- DELETE /api/donors/{id} - optional. Avoid deleting if donations exist; use soft delete only if added.

7. Inventory module

Inventory stores blood stock per blood category. Each blood category should have only one inventory row. Status is calculated from quantity, never selected manually.

Class	Field	UI / input type	Source / options	Validation / annotations
InventoryRequest	categoryId	Blood group dropdown	GET /api/categories/active	@NotNull. Service: category must exist. Unique inventory per category.
InventoryRequest	quantity	Number input	Admin typed	@NotNull, @Min(0).
InventoryRequest	notes	Text area	Admin typed	@Size(max=200). Optional.
Inventory entity	id	Generated	N/A	@Id, @GeneratedValue(strategy=IDENTITY).
Inventory entity	category	Relationship	blood_categories.id	@OneToOne, @JoinColumn(name="category_id", nullable=false, unique=true).
Inventory entity	quantity	Column	N/A	@Column(nullable=false). Must be >= 0 by service/request.
Inventory entity	status	No UI input	Auto from quantity	@Enumerated(EnumType.STRING), @Column(nullable=false,length=20).
Inventory entity	updatedAt	Generated/service set	N/A	Use LocalDateTime. Update when quantity changes.

Inventory status calculation

Item	Decision
quantity = 0	OUT_OF_STOCK
quantity between 1 and 3	LOW_STOCK
quantity >= 4	AVAILABLE
Common mistake	Do not accept status from frontend request. Calculate it in InventoryService every time quantity changes.

Inventory endpoints

- GET /api/inventory - list stock.
- GET /api/inventory/{id} - detail.
- POST /api/inventory - create stock row for category if not already created.
- PUT /api/inventory/{id} - update quantity/notes and recalculate status.
- GET /api/inventory/low-stock - optional helper endpoint.

8. Donation module

Donation records are history records. When a donation is saved, inventory must increase for the same blood category. Use @Transactional because donation save and inventory update must succeed together.

Class	Field	UI / input type	Source / options	Validation / annotations
DonationRequest	donorId	Donor dropdown/search	GET /api/donors	@NotNull. Service: donor must exist.
DonationRequest	categoryId	Auto-filled or dropdown	Prefer donor.category from selected donor; if dropdown, GET /api/categories/active	@NotNull. Service: category must match donor category.
DonationRequest	quantity	Number input	Admin typed	@NotNull, @Min(1). Normally 1 unit.
DonationRequest	donationDate	Date picker	Admin typed	@NotNull, @PastOrPresent.
DonationRequest	location	Text input	Admin typed	@NotBlank, @Size(min=2,max=120).
DonationRequest	notes	Text area	Admin typed	@Size(max=250). Optional.
Donation entity	donor	Relationship	donors.id	@ManyToOne, @JoinColumn(name="donor_id", nullable=false).
Donation entity	category	Relationship	blood_categories.id	@ManyToOne, @JoinColumn(name="category_id", nullable=false).
Donation entity	status	No UI input	RECORDED only	@Enumerated(EnumType.STRING), @Column(nullable=false). Default RECORDED.

Donation service flow

- Find donor by donorId. If not found -> ResourceNotFoundException 404.
- Find category by categoryId. If not found -> ResourceNotFoundException 404.
- Check category matches donor.getCategory(). If mismatch -> BadRequestException 400.
- Validate quantity > 0 and donationDate is not future.
- Save donation with status RECORDED.
- Increase inventory.quantity for the same category and recalculate inventory.status.
- Return DonationResponse. Do not confuse Donation with Announcement.

9. Announcement module

Announcement is a blood need/request post, not blood stock. It tells users which blood group is needed.

Class	Field	UI / input type	Source / options	Validation / annotations
AnnouncementRequest	title	Text input	Admin typed	@NotBlank, @Size(min=5,max=100).
AnnouncementRequest	categoryId	Blood group dropdown	GET /api/categories/active	@NotNull. Service: category must exist and active=true.
AnnouncementRequest	quantityNeeded	Number input	Admin typed	@NotNull, @Min(1).
AnnouncementRequest	urgency	Dropdown	NORMAL, URGENT, EMERGENCY	@NotNull. Store urgency enum/string.
AnnouncementRequest	location	Text input	Admin typed	@NotBlank, @Size(min=2,max=120).
AnnouncementRequest	description	Text area	Admin typed	@Size(max=300). Optional.
AnnouncementStatusRequest	status	Dropdown	ACTIVE, FULFILLED, CANCELLED	@NotNull. Used only for status update.
Announcement entity	category	Relationship	blood_categories.id	@ManyToOne, @JoinColumn(name="category_id", nullable=false).
Announcement entity	status	Default/update field	Default ACTIVE	@Enumerated(EnumType.STRING), @Column(nullable=false).

Announcement status rules

Item	Decision
Create	status must be ACTIVE automatically. Frontend should not send status on create.
Update	Admin can change status to FULFILLED or CANCELLED. Keep record for reports.
Common mistake	Do not reduce inventory automatically when announcement is created. Announcement is need/request, not stock.

Announcement endpoints

- POST /api/announcements - create active announcement.
- GET /api/announcements - list announcements.
- GET /api/announcements/active - public/home page active requests.
- PUT /api/announcements/{id} - update content.
- PATCH /api/announcements/{id}/status - update status only.

10. Dashboard and Report modules

Dashboard and Report do not need entity tables. They read existing repositories and return response DTOs.

Module/class	Field/input	UI type	Source/options	Validation/rule
DashboardRequest	None	No input	GET /api/dashboard	Read-only summary. No request class needed.
DashboardResponse	totalDonors	Response	DonorRepository.count()	Number.
DashboardResponse	totalDonations	Response	DonationRepository.count()	Number.
DashboardResponse	totalBloodUnits	Response	sum inventory quantity	Number.
DashboardResponse	activeAnnouncements	Response	Announcement status ACTIVE count	Number.
DashboardResponse	lowStockGroups	Response	Inventory status LOW_STOCK count	Number.
DashboardResponse	outOfStockGroups	Response	Inventory status OUT_OF_STOCK count	Number.
ReportFilterRequest	reportType	Dropdown	DONOR, INVENTORY, DONATION, ANNOUNCEMENT	@NotNull.
ReportFilterRequest	fromDate/toDate	Date pickers	User typed dates	Optional. Service: fromDate must be <= toDate.
ReportFilterRequest	categoryId	Blood group dropdown	GET /api/categories/active	Optional. Use categoryId, display bloodGroup.
ReportFilterRequest	status	Dropdown optional	Depends on reportType	Optional. Service validates status belongs to report type.

Report status filter options

Item	Decision
DONOR report	Usually no status filter unless you add donor status later.
INVENTORY report	AVAILABLE, LOW_STOCK, OUT_OF_STOCK.
DONATION report	RECORDED only.
ANNOUNCEMENT report	ACTIVE, FULFILLED, CANCELLED.

Report output can be JSON only. PDF export is not required by the project specification; final submission needs project report and optional schema/ERD separately.

11. JPA relationships and database constraints

These relationships create primary key and foreign key constraints through Hibernate/JPA. Do not manually create different table structures in pgAdmin.

Relationship	JPA annotation	Database result	Meaning
User -> Donor	@OneToOne in Donor: user; @JoinColumn(name="user_id", unique=true, nullable=false)	donors.user_id FK -> users.id; unique	One account can have one donor profile.
BloodCategory -> Donor	@ManyToOne in Donor: category; @JoinColumn(name="category_id", nullable=false)	donors.category_id FK -> blood_categories.id	Donor has one blood group/category.
BloodCategory -> Inventory	@OneToOne in Inventory: category; @JoinColumn(name="category_id", unique=true, nullable=false)	inventory.category_id FK -> blood_categories.id; unique	One inventory stock row per blood group.
Donor -> Donation	@ManyToOne in Donation: donor; @JoinColumn(name="donor_id", nullable=false)	donations.donor_id FK -> donors.id	One donor can have many donations.
BloodCategory -> Donation	@ManyToOne in Donation: category; @JoinColumn(name="category_id", nullable=false)	donations.category_id FK -> blood_categories.id	Donation belongs to a blood group.
BloodCategory -> Announcement	@ManyToOne in Announcement: category; @JoinColumn(name="category_id", nullable=false)	announcements.category_id FK -> blood_categories.id	Announcement requests a specific blood group.

Entity order to build and push first

1. User
2. BloodCategory
3. Donor
4. Inventory
5. Donation
6. Announcement
7. Repositories for all entities
8. Exceptions
9. Services/controllers

Local DB rule for group work: each member can use local PostgreSQL, but the same Git code/entities must be used. Entities are the shared database contract. Database name: blood_donation_db.

12. Exceptions, HTTP status codes, and validation

Exceptions make backend errors consistent for React. Services throw exceptions; GlobalExceptionHandler converts them to ErrorResponse JSON.

Exception	HTTP status	When to use	Example
BadRequestException	400 BAD REQUEST	User input is invalid or business rule failed.	Quantity <= 0, future donation date, blood group mismatch.
ResourceNotFoundException	404 NOT FOUND	Requested entity does not exist.	Donor not found, category not found, announcement not found.
DuplicateResourceException	409 CONFLICT	Unique data already exists.	Email already exists, blood group already exists, donor profile already exists.
Exception	500 INTERNAL SERVER ERROR	Unexpected server error.	NullPointerException, database connection issue, unknown bug.

ErrorResponse fields

Item	Decision
status	HTTP code: 400, 404, 409, 500.
message	User-friendly error message such as "Email already exists".
path	API path where error happened, such as /api/auth/register.
timestamp optional	You may add LocalDateTime timestamp if you want, but status/message/path are enough.

Controller validation rule

```
@PostMapping
public ResponseEntity create(@Valid @RequestBody RequestDto request) {
    return ResponseEntity.status(HttpStatus.CREATED).body(service.create(request));
}
```

Validation must be on request DTOs. Entity @Column constraints are database mapping, not a replacement for frontend/backend input validation.

13. Two-person work plan: Isse and Yasmin

The work is not fully parallel at the beginning. Isse must first push the database contract: entities, repositories, exceptions, status enums/rules. After that, both can work in parallel.

Person	First tasks	Can work parallel after contract?	Depends on
Isse	Application config, User, Auth, Security JWT, Category, Exceptions, core entity/repository contract, integration review.	Yes	Must push entities/repositories first so Yasmin can use them.
Yasmin	Donor, Inventory, Donation, Announcement, Dashboard, Report. Can start DTOs and controller skeletons before contract.	Yes, after pulling Isse contract	Needs User, BloodCategory, Donor, Inventory repositories and status rules.

Recommended Git sequence

```
Isse:
git checkout -b feature/core-contract
# add entities, repositories, exceptions, enums/rules
git add .
git commit -m "Add core JPA entities repositories and rules"
git push origin feature/core-contract
```

```
After merge/pull, Yasmin:
git pull origin master
# or git pull origin dev, depending on your branch
```

What Yasmin can do before Isse finishes

- Create request and response DTOs for donor, inventory, donation, announcement.
- Create controller method skeletons with endpoints.
- Prepare React forms and dropdown UI fields.
- Do not finalize service logic that saves Donation or updates Inventory until entities and repositories are pulled.

14. Final correctness checklist before coding

Area	Must be true before coding continues
Requirement alignment	Backend uses Spring Boot 3.x, Java 17+, Spring Data JPA + Hibernate, Spring Security JWT, PostgreSQL, Validation, ReactJS + Axios.
Structure	Every main module has controller/service/repository/entity/request/response except auth, dashboard, report, security, exception as defined.
Auth	Auth uses UserRepository. It returns JWT. Password is encoded. Password never appears in response.
Roles	Use ROLE_ADMIN and ROLE_USER. Protect admin-only APIs using SecurityConfig or @PreAuthorize.
Blood group dropdown	Source is GET /api/categories/active. Frontend displays bloodGroup but sends categoryId.
Inventory status	No dropdown. Service calculates OUT_OF_STOCK, LOW_STOCK, AVAILABLE from quantity.
Donation	Status is RECORDED only. Saving donation increases inventory in same transaction.
Announcement	Create default ACTIVE. Status update options are ACTIVE, FULFILLED, CANCELLED.
Dashboard	Read-only summary. No request input. No entity table needed.
Report	Read-only filtered JSON. Request uses reportType, dates, categoryId, optional status.
Validation	Use @Valid in controllers and Bean Validation annotations on request classes.
Database	Database name: blood_donation_db. Tables are generated by JPA/Hibernate from entities.

Final note: If a field is fixed enum data such as role, gender, urgency, or reportType, the frontend can hardcode the dropdown options, but backend must still validate. If a field comes from database such as blood category or donor list, the dropdown/search must call backend APIs.